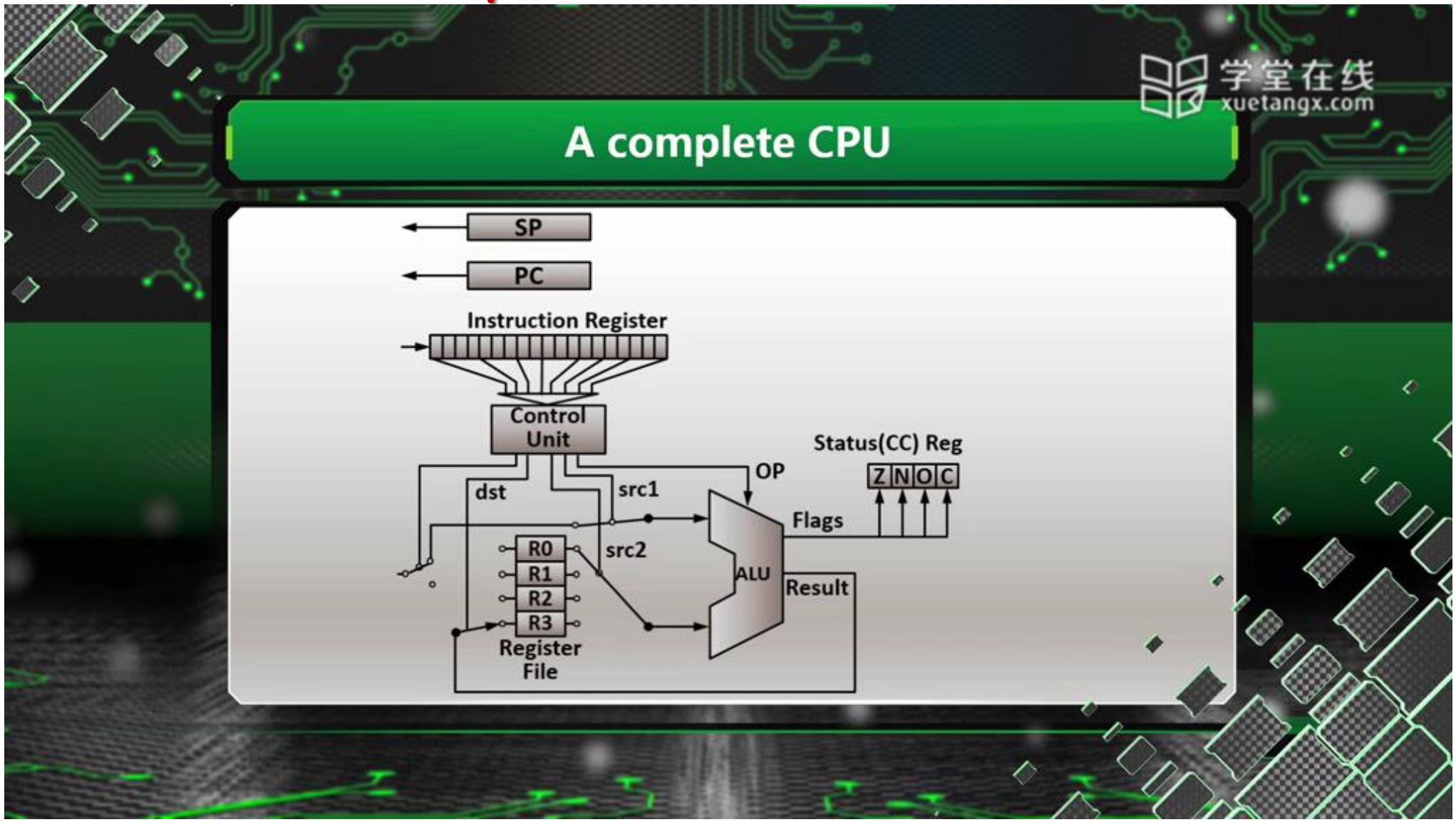
EE114: Introduction to Embedded Systems

Dr. Tao Wu 吴涛
wutao@shanghaitech.edu.cn
Office: SIST Bldg 3-326
CourseWeb: http://10.15.25.5/
Website: http://small.shanghaitech.edu.cn

A Complete CPU

- MCU Structure
  - The CPU
    - Registers file （寄存器组）
    - Instruction latch/decode/branch （指令）
    - Stack（栈）
    - Interrupt（中断）
    - Clock/Reset（时钟/复位）
  - Memory
  - Bus
  - Peripheral

A Complete CPU – Video 23'

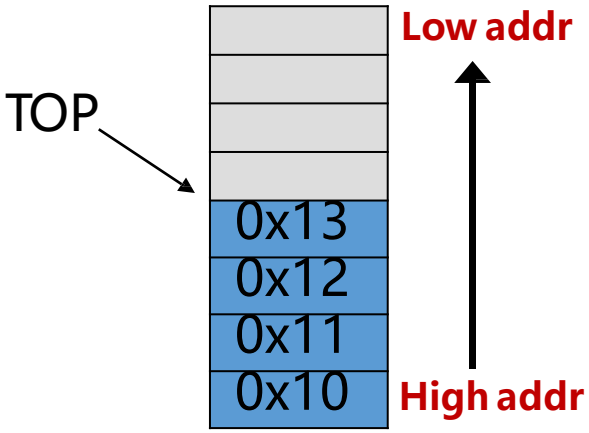


Stack 栈

- 堆栈是一段连续的存储器空间
- 堆栈按照**后入先出**的方式工作(**Last In First Out**)
- 只能向/从堆栈的**顶部**加入或取出数据
- 顶端位置通过堆栈指针寄存器确定(SP，Stack Pointer)
- 两种基本的操作方式:

  推入PUSH:　将内容加入到堆栈顶端

  取出PULL：　将堆栈顶端的内容取出



0x10 is the first pushed item
0x13 is the last pushed item

- 对于**大多数CPU**而言，“**顶端**”是指低位的地址空间；但 **8051**是**向上生长型**的堆栈
- 堆栈容量不超过（256-8）字节，单片机复位后SP的值为**07H**，**SP是入栈递增**
- 汇编程序可以使用堆栈来保存局部变量，寄存器值
- C语言编译器使用堆栈来完成参数传递和返回值传递 -> C语言的函数调用
- CPU硬件 使用堆栈 来保存返回地址和寄存器上下文 (register context)

Stack vs. Heap

- 堆Heap：是一个进程开启以后，系统分配给它的空间，一般这个空间是全局的，系统中所有动态分配的对象（比如指针）都是在这个空间上分配。 堆里的数据是有数据结构的，其空间占用是不连续的。在没有OS的嵌入式系统中，通常不使用堆。

- 栈Stack：是一个线程维护的一个先进后出的数据结构，主要用于静态变量的 分配及维护程序各个函数调用时使用。栈里的内容是没有数据结构的，其空间占用是连续的。栈是微处理器的必要特性，是支持高级语言编程的前提。

Run Example

S12 CPU (16 bits)

```
          0x3005:  NOP
          0x3006:  LDS $2000
          0x3008:  PSHA
          0x3009:  PSHB
          0x300A: JSR SubFunc
          0x300C:  PULLA
          0x300D:  PULLB
           ..............
SubFunc: 0x4050: NOP
          0x4052: RTS
```

Register file **after** this instruction

0x1FFB	XX
0x1FFC	XX
0x1FFD	XX
0x1FFE	XX
0x1FFF	XX

SP = undefined
PC = **0x3006**
A = 0x34
B = 0x56

current executed instruction
current stack pointer

NOP: no operation, do nothing
LDS : load stack pointer with a value
PSHx: push the contains of register 'x' into stack
PULLx: pull from the stack and put in register 'x'
JSR : jump to subroutine
RTS : return from subroutine

Run Example

```
            0x3005:  NOP
→           0x3006:  LDS $2000
            0x3008:  PSHA
            0x3009:  PSHB
            0x300A: JSR SubFunc
            0x300C:  PULLA
            0x300D:  PULLB
            ..............
SubFunc: 0x4050: NOP
            0x4052: RTS
```

→ current executed instruction
← current stack pointer

Register file **after** this instruction

0x1FFB	XX
0x1FFC	XX
0x1FFD	XX
0x1FFE	XX
0x1FFF	XX ←

SP = **0x2000**
PC = **0x3008**
A = 0x34
B = 0x56

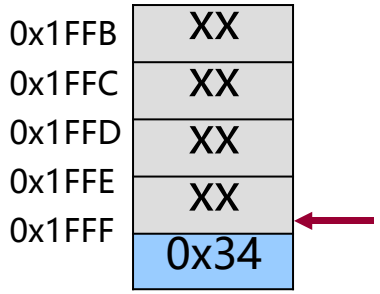
NOP: no operation, do nothing
LDS : load stack pointer with a value
PSHx: push the contains of register 'x' into stack
PULLx: pull from the stack and put in register 'x'
JSR : jump to subroutine
RTS : return from subroutine

Run Example

```
          0x3005:  NOP
          0x3006:  LDS $2000
→         0x3008:  PSHA
          0x3009:  PSHB
          0x300A: JSR SubFunc
          0x300C:  PULLA
          0x300D:  PULLB
          ..............
SubFunc: 0x4050: NOP
          0x4052: RTS
```

→ current executed instruction
← current stack pointer

Register file **after** this instruction



SP = **0x1FFF**
PC = **0x3009**
A = 0x34
B = 0x56

NOP: no operation, do nothing
LDS : load stack pointer with a value
PSHx: push the contains of register 'x' into stack
PULLx: pull from the stack and put in register 'x'
JSR : jump to subroutine
RTS : return from subroutine

Run Example

```
            0x3005:  NOP
            0x3006:  LDS $2000
            0x3008:  PSHA
→           0x3009:  PSHB
            0x300A: JSR SubFunc
            0x300C:  PULLA
            0x300D:  PULLB
            ..............
SubFunc: 0x4050: NOP
            0x4052: RTS
```

→ current executed instruction
← current stack pointer

Register file **after** this instruction

0x1FFB	XX
0x1FFC	XX
0x1FFD	XX
0x1FFE	0x56 ←
0x1FFF	0x34

SP = **0x1FFE**
PC = **0x300A**
A = 0x34
B = 0x56

NOP: no operation, do nothing
LDS : load stack pointer with a value
PSHx: push the contains of register 'x' into stack
PULLx: pull from the stack and put in register 'x'
JSR : jump to subroutine
RTS : return from subroutine

Run Example

```
          0x3005:  NOP
          0x3006:  LDS $2000
          0x3008:  PSHA
          0x3009:  PSHB
→         0x300A: JSR SubFunc
          0x300C:  PULLA
          0x300D:  PULLB
          ..............
SubFunc: 0x4050: NOP
          0x4052: RTS
```

→ current executed instruction
← current stack pointer

Register file **after** this instruction

0x1FFB	XX
0x1FFC	0x30 ←
0x1FFD	0x0C
0x1FFE	0x56
0x1FFF	0x34

SP = **0x1FFC**
PC = **0x4050**
A = 0x34
B = 0x56

NOP: no operation, do nothing
LDS : load stack pointer with a value
PSHx: push the contains of register 'x' into stack
PULLx: pull from the stack and put in register 'x'
JSR : jump to subroutine
RTS : return from subroutine

Run Example

```
0x3005:  NOP
0x3006:  LDS $2000
0x3008:  PSHA
0x3009:  PSHB
0x300A: JSR SubFunc
0x300C:  PULLA
0x300D:  PULLB
.............
SubFunc: 0x4050: NOP
         0x4052: RTS
```

Register file **after** this instruction

0x1FFB	XX
0x1FFC	0x30
0x1FFD	0x0C
0x1FFE	0x56
0x1FFF	0x34

SP = **0x1FFC**
PC = **0x4052**
A = 0x34
B = 0x56

current executed instruction
current stack pointer

NOP: no operation, do nothing
LDS : load stack pointer with a value
PSHx: push the contains of register 'x' into stack
PULLx: pull from the stack and put in register 'x'
JSR : jump to subroutine
RTS : return from subroutine

Run Example

```
            0x3005:  NOP
            0x3006:  LDS $2000
            0x3008:  PSHA
            0x3009:  PSHB
            0x300A: JSR SubFunc
            0x300C:  PULLA
            0x300D:  PULLB
             ..............
SubFunc: 0x4050: NOP
         0x4052: RTS
```

Register file **after** this instruction

0x1FFB	XX
0x1FFC	0x30
0x1FFD	0x0C
0x1FFE	0x56
0x1FFF	0x34

SP = **0x1FFE**
PC = **0x300C**
A = 0x34
B = 0x56

current executed instruction
current stack pointer

NOP: no operation, do nothing
LDS : load stack pointer with a value
PSHx: push the contains of register 'x' into stack
PULLx: pull from the stack and put in register 'x'
JSR : jump to subroutine
RTS : return from subroutine

Run Example

```
          0x3005:  NOP
          0x3006:  LDS $2000
          0x3008:  PSHA
          0x3009:  PSHB
          0x300A: JSR SubFunc
→         0x300C:  PULLA
          0x300D:  PULLB
          ..............
SubFunc: 0x4050: NOP
          0x4052: RTS
```

→ current executed instruction
← current stack pointer

Register file **after** this instruction

0x1FFB	XX
0x1FFC	0x30
0x1FFD	0x0C
0x1FFE	0x56
0x1FFF	0x34 ←

SP = **0x1FFF**
PC = **0x300D**
A = **0x56**
B = 0x56

NOP: no operation, do nothing
LDS : load stack pointer with a value
PSHx: push the contains of register 'x' into stack
PULLx: pull from the stack and put in register 'x'
JSR : jump to subroutine
RTS : return from subroutine

Run Example

```
          0x3005:  NOP
          0x3006:  LDS $2000
          0x3008:  PSHA
          0x3009:  PSHB
          0x300A: JSR SubFunc
          0x300C:  PULLA
→         0x300D:  PULLB
           ..............
SubFunc: 0x4050: NOP
          0x4052: RTS
```

→ current executed instruction
← current stack pointer

Register file **after** this instruction

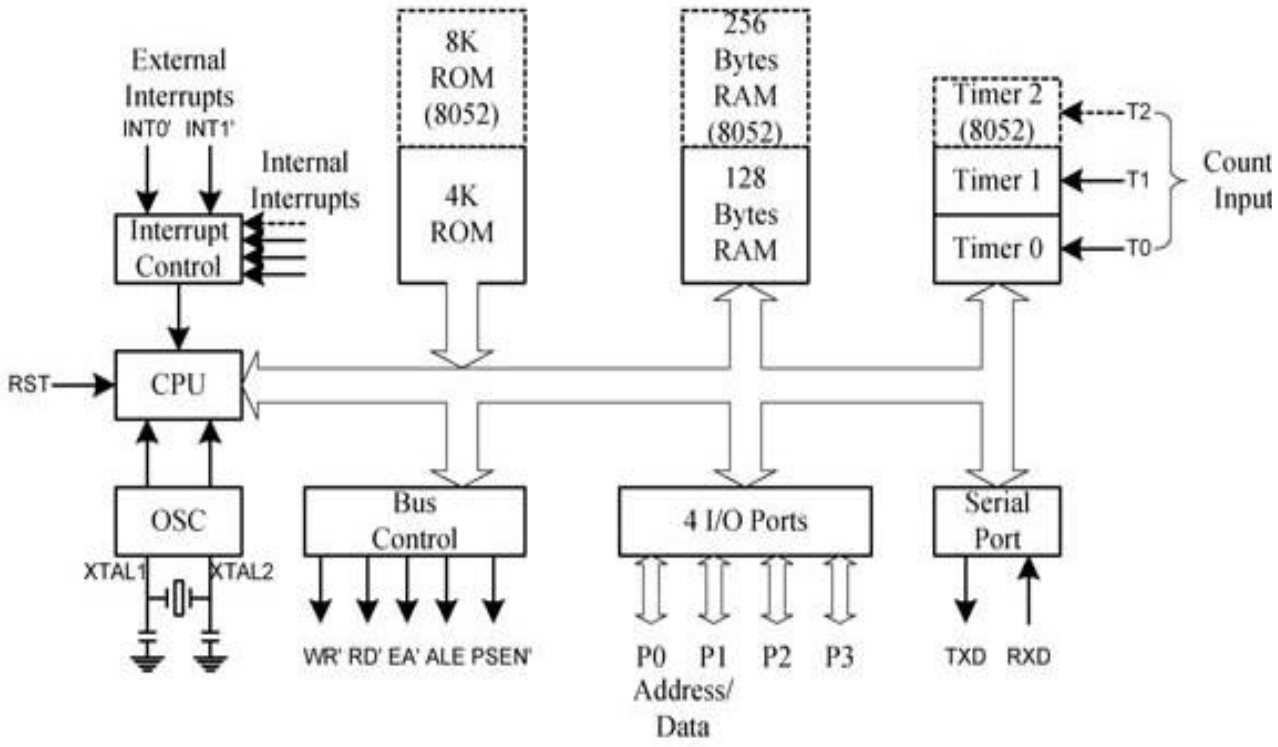
0x1FFB	XX
0x1FFC	0x30
0x1FFD	0x0C
0x1FFE	0x56
0x1FFF	0x34 ←

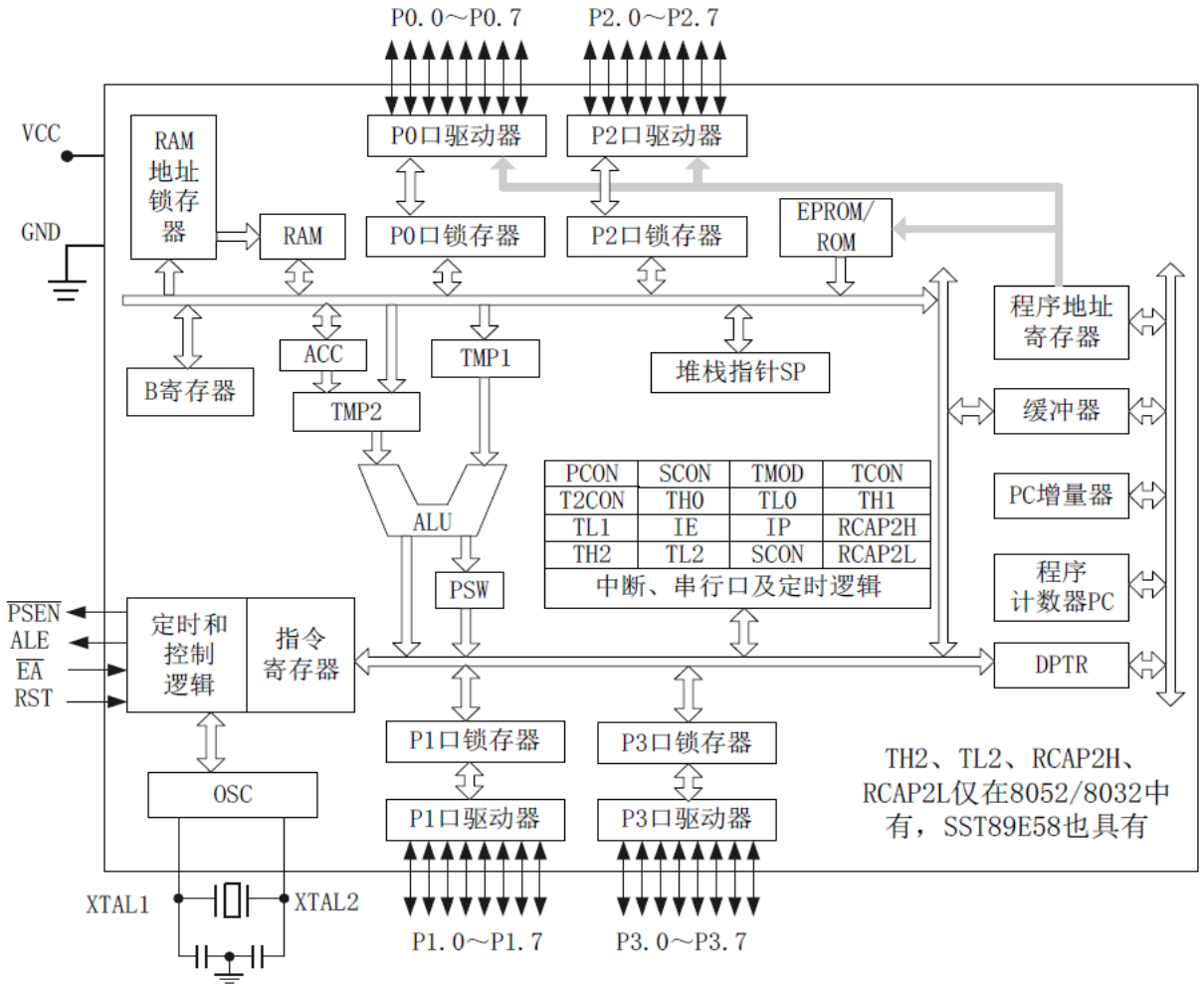
SP = **0x2000**
PC = **0x300E**
A = 0x56
B = **0x34**

NOP: no operation, do nothing
LDS : load stack pointer with a value
PSHx: push the contains of register 'x' into stack
PULLx: pull from the stack and put in register 'x'
JSR : jump to subroutine
RTS : return from subroutine

8051 Review

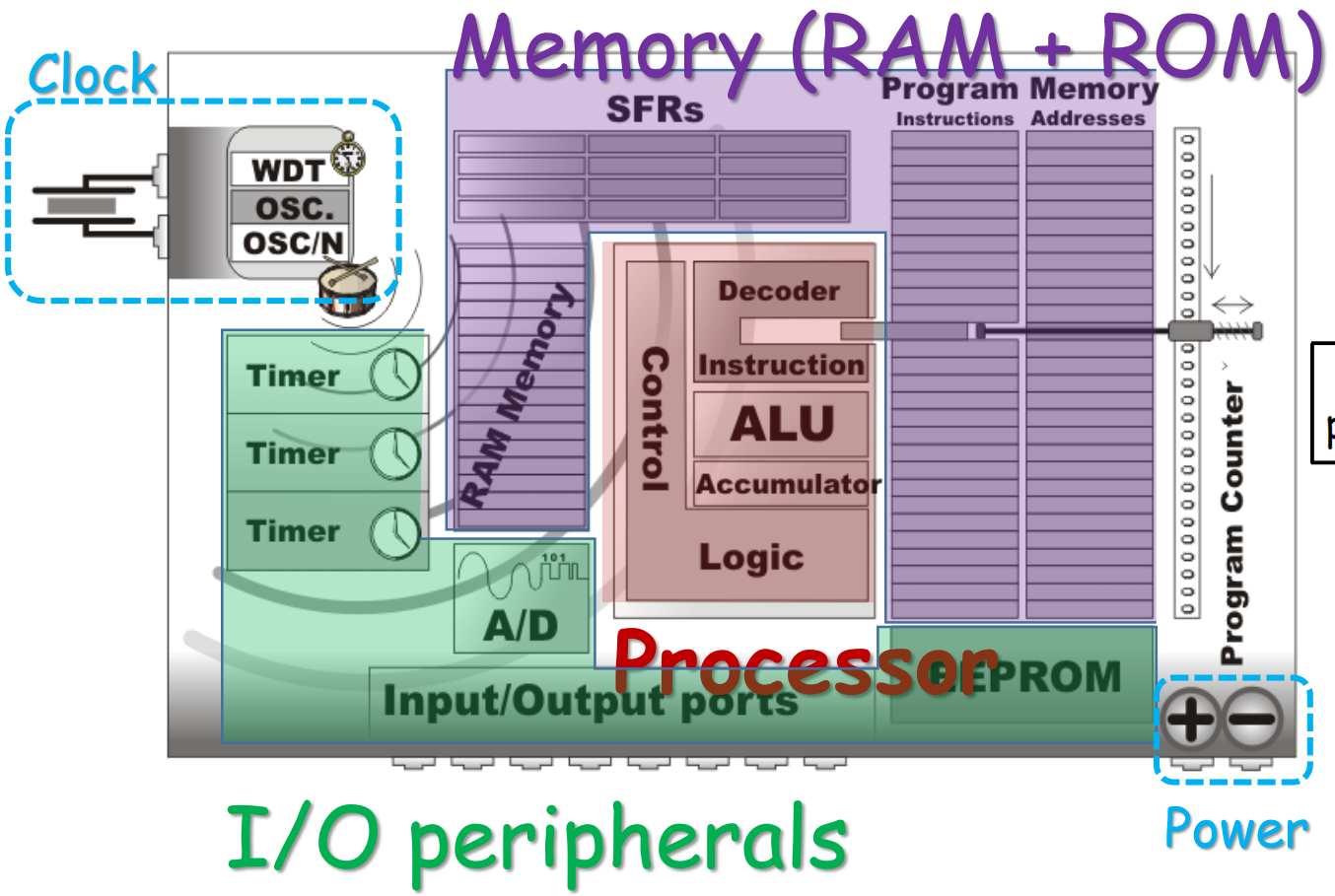
MCS-8051单片机主要功能方框图

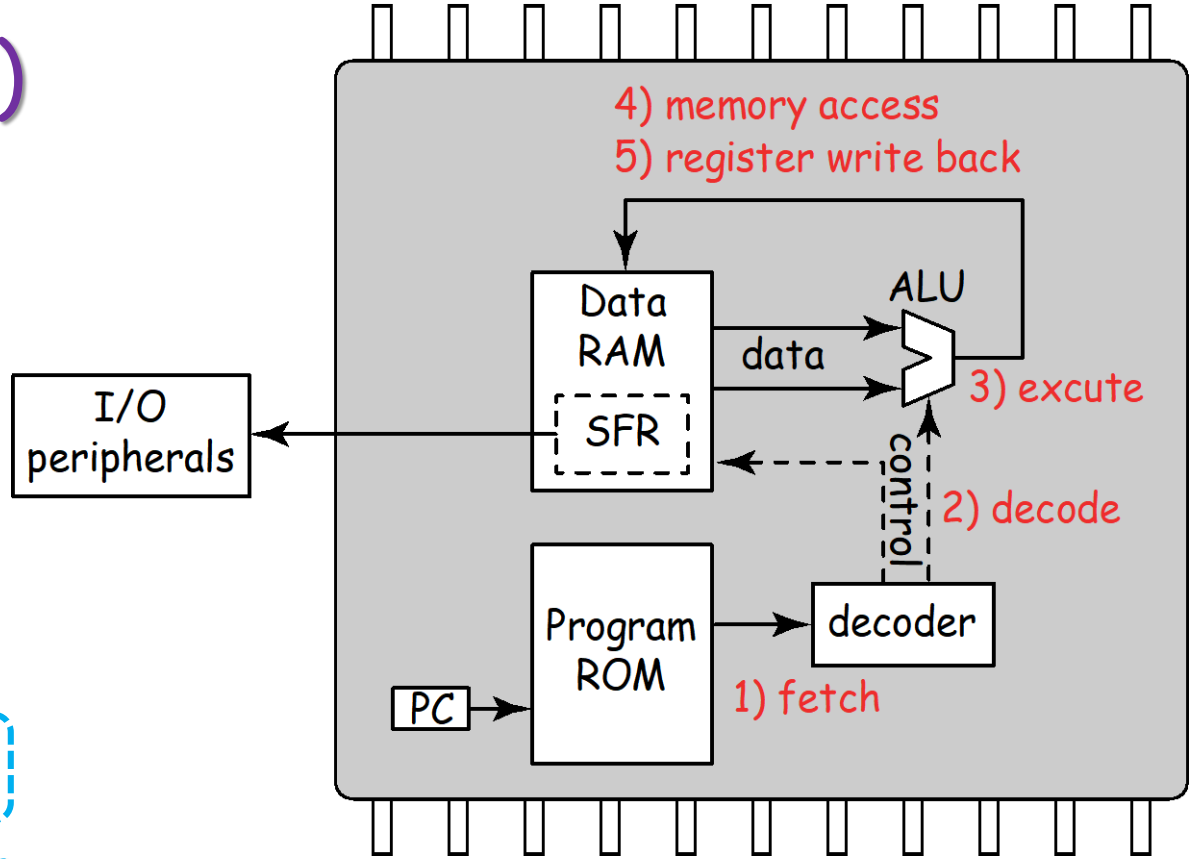




How it runs

Key terms: ALU, R0～R7, Acc, B, PSW, DPTR, SP, PC





SFR: special function register

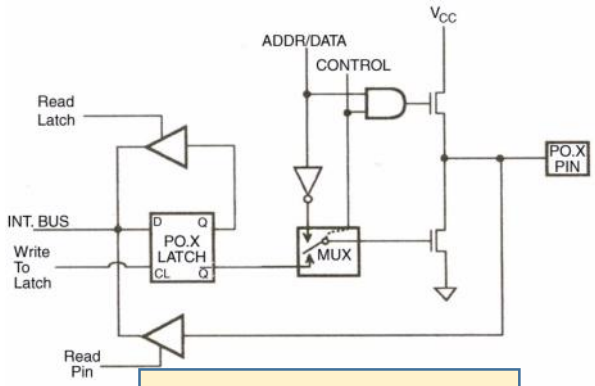
(for recording the state, controlling I/O, etc.)

状态寄存器PSW的格式如下：

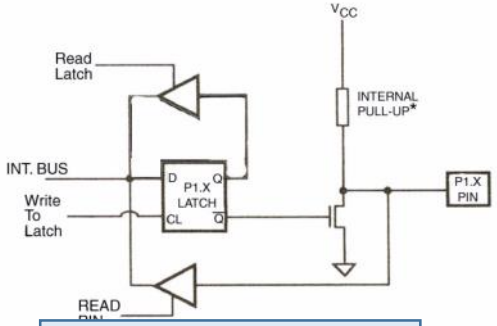
D7	D6	D5	D4	D3	D2	D1	D0 ←位地址	
CY	AC	F0	RS1	RS0	OV	/	P	D0H

↑ 位累加器

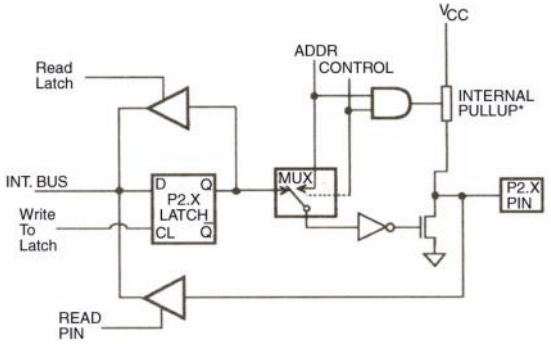
8051 I/O (hardware)



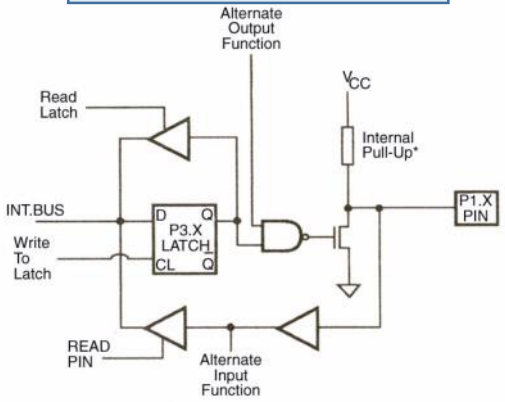
PORT 0 BIT



PORT 1 BIT



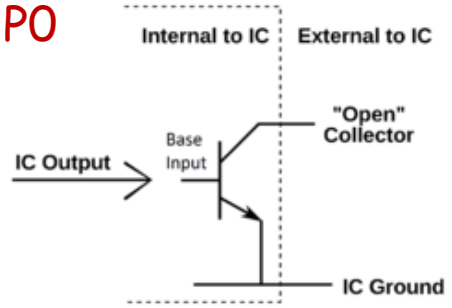
PORT 2 BIT



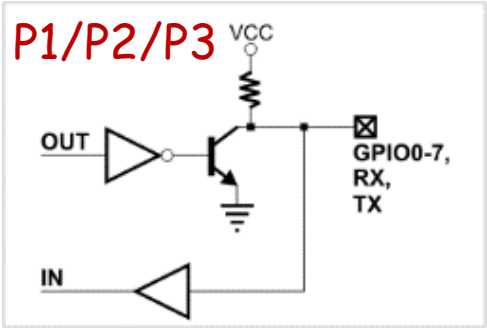
PORT 3 BIT

- Ports 0 has open-drain outputs ("true" bidirectional).
  - external memory access
  - Drive devices
  - open-collector output can connect to a single line for wired logic connection
- Ports 1, 2, and 3 have internal pull-ups (quasi-bidirectional ports).
  - To "measure" an external signal applied to a GPIO line, set this line to HIGH first, then read the state of the pin

PO



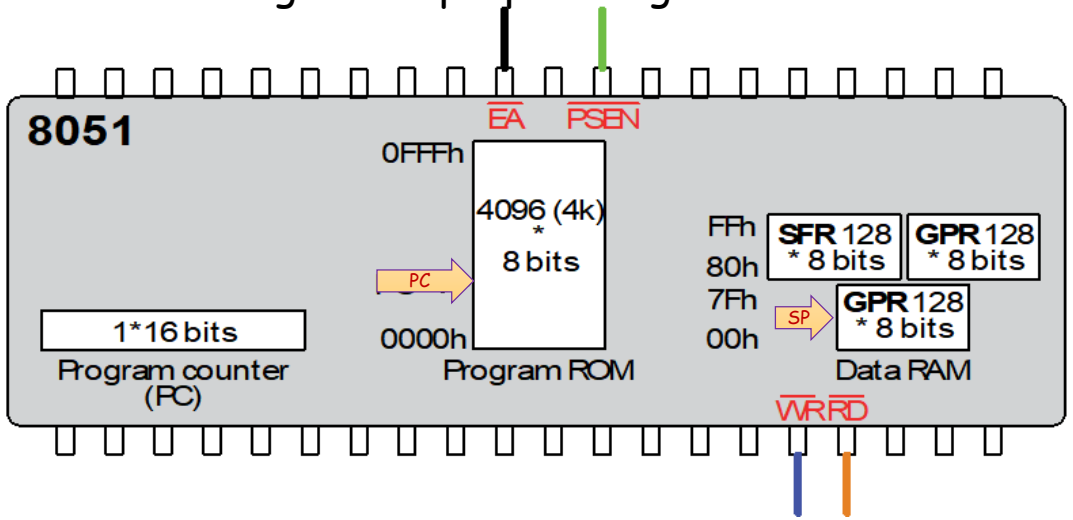
P1/P2/P3

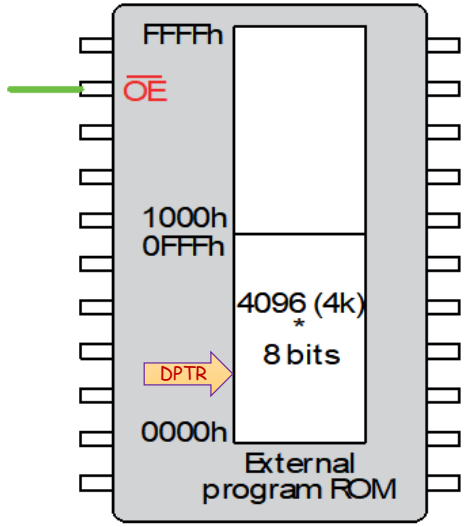


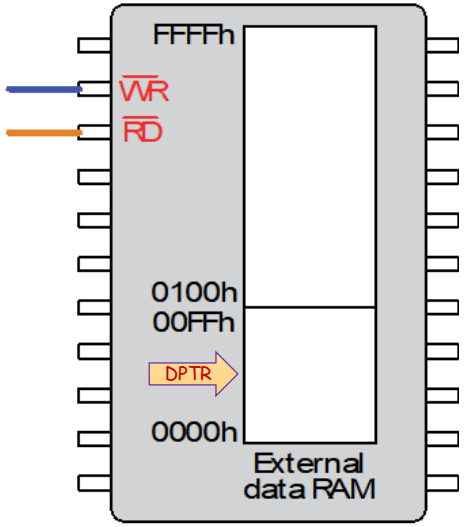
8051 Memory Mapping

SFR: special function register

GPR: general-purpose register







序号	寻址方式	相应的存储器空间及寄存器
1	寄存器寻址	R0～R7、ACC、CY(位寻址)、DPTR
2	直接寻址	内部 RAM（0～128B）、SFR
3	寄存器间接寻址	内部 RAM（@Ri、SP）、外部 RAM（@Ri,@DPIR）
4	立即寻址	程序存储立即数
5	基址+变址寄存器的间接寻址	程序存储器（@DPTR+A，@PC+A）
6	相对寻址	以 PC 的当前值为基地址+ rel=有效地址。转移范围：PC 当前值的-128～+127
7	位寻址	对内部 RAM 或 SFR 中有定义的单元进行位寻址

SFR of original 8051

Red address: bit addresses

Black address: byte addresses

Data and stack pointers

CPU

Interrupt control

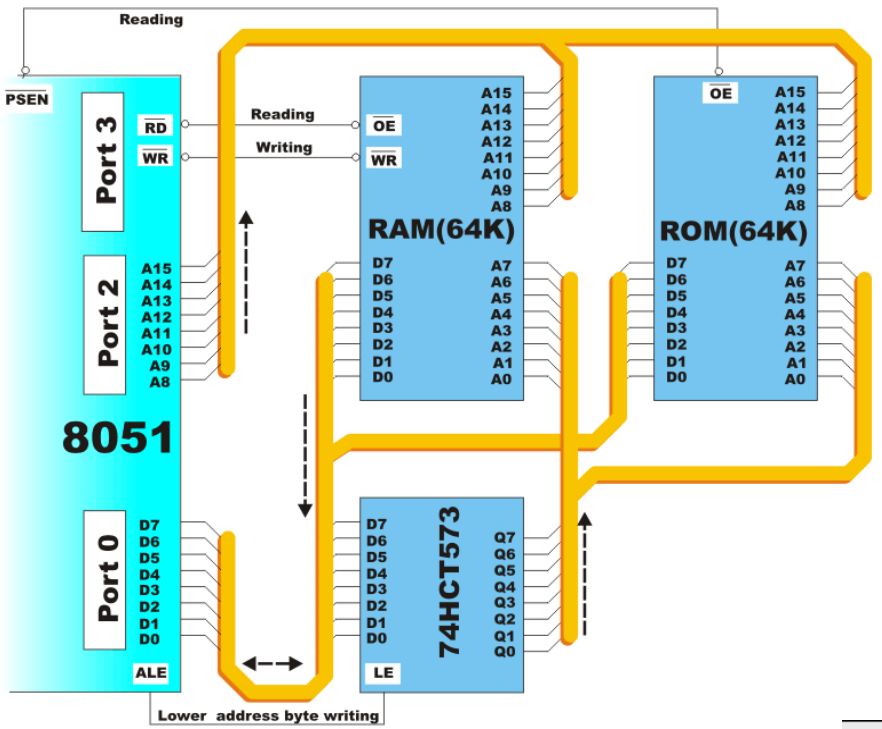
I/O ports

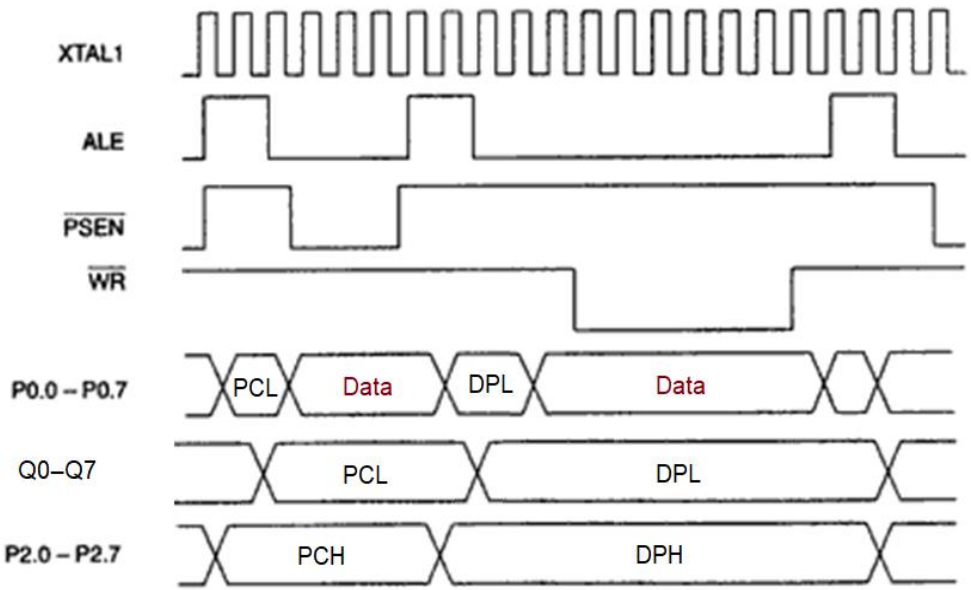
UART

Timers

F8h-FFh	(F8h-FFh)								128 SFR
F0h-F7H	**B** (F0h-F7H)								
E8h-EFh	(E8h-EFh)								
E0h-E7h	**ACC** (E0h-E7h)								
D8h-DFh	(D8h-DFh)								
D0h-D7h	**PSW** (D0h-D7h)								
C8h-CFh	(C8h-CFh)								
C0h-C7h	(C0h-C7h)								
B8h-BFh	**IP** (B8h-BFh)								
B0h-B7h	**P3** (B0h-B7h)								
A8h-AFh	**IE** (A8h-AFh)								
A0h-A7h	**P2** (A0h-A7h)								
98h-9Fh	**SCON** (98h-9Fh)	**SBUF**							
90h-97h	**P1** (90h-97h)								
88h-8Fh	**TCON** (88h-8Fh)	**TMOD**	**TL0**	**TL1**	**TH0**	**TH1**			
80h-87h	**P0** (80h-87h)	**SP**	**DPL**	**DPH**				PCON	
30h-7Fh									80 General purpose registers
20h-2Fh	40h-47h	48h-4Fh	50h-57h	58h-5Fh	60h-67h	68h-6Fh	70h-77h	78h-7Fh	16 bit-addressable
	00h-07h	08-0Fh	10h-17h	18h-1Fh	20h-27h	28-2Fh	30h-37h	38h-3Fh	
18h-1Fh	R0	R1	R2	R3	R4	R5	R6	R7	Bank 3
10h-17h	R0	R1	R2	R3	R4	R5	R6	R7	Bank 2
08h-0Fh	R0	R1	R2	R3	R4	R5	R6	R7	Bank 1
00h-07h	R0	R1	R2	R3	R4	R5	R6	R7	Bank 0

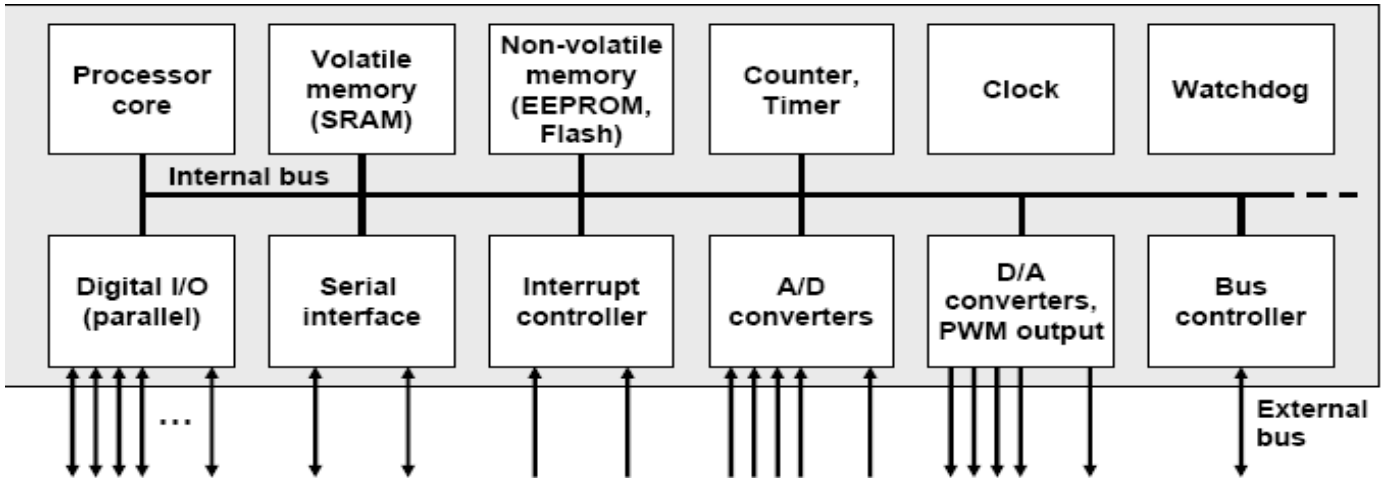
Connecting external RAM/ROM





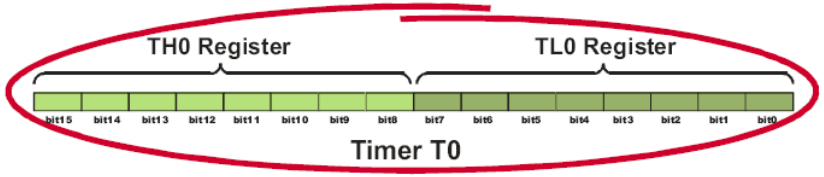
Parallel RAM/ROM extensions are rarely used nowadays;
But the logic & sequence apply to any peripheral or external chips

MCU with more integrated peripherals

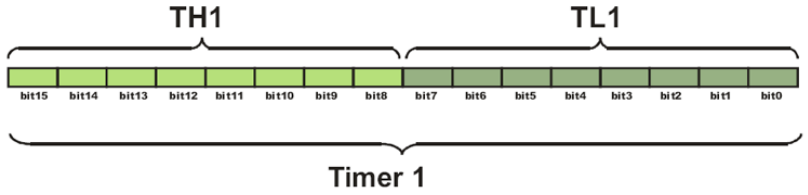


8051 timer/counter (hardware)

- There are two 16 bits timers/counters in 8051
  - Timer 0



  - Timer 1

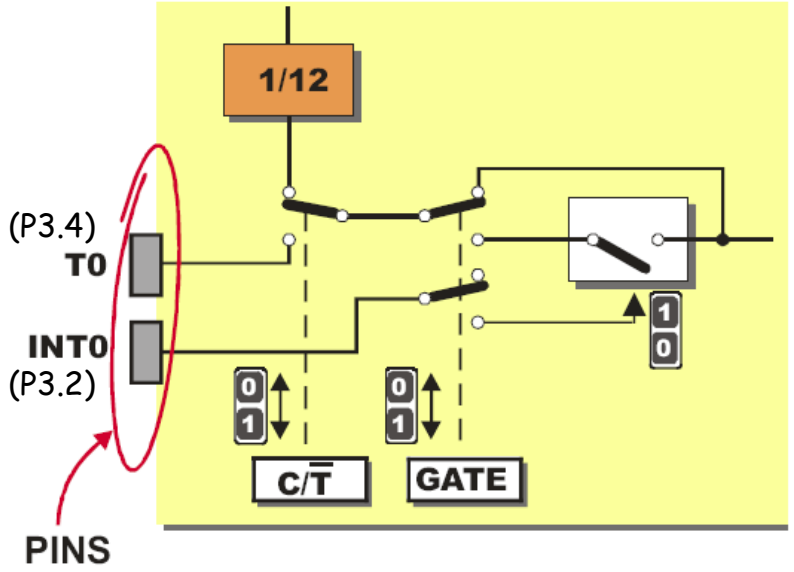


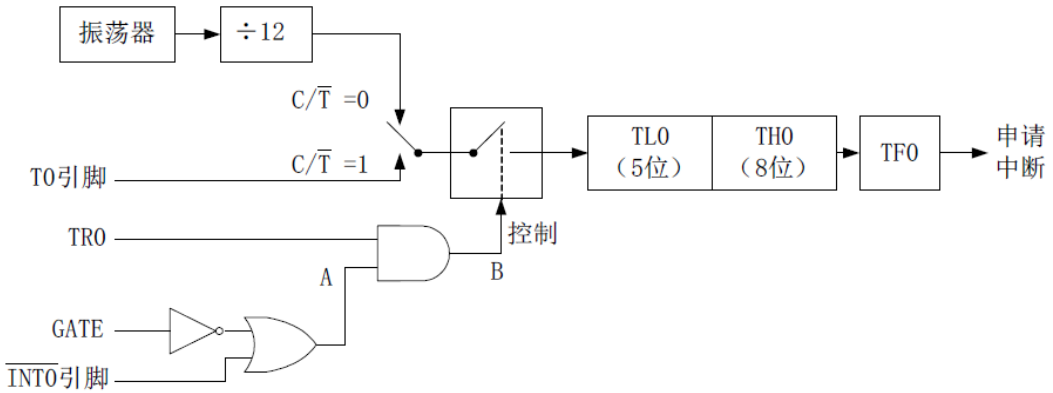
- Related SFR

	0	0	0	0	0	0	0	0	Value after reset
TMOD	GATE1	C/T1	T1M1	T1M0	GATE0	C/T0	T0M1	T0M0	Bit name
	bit7	bit6	bit5	bit4	bit3	bit2	bit1	bit0	

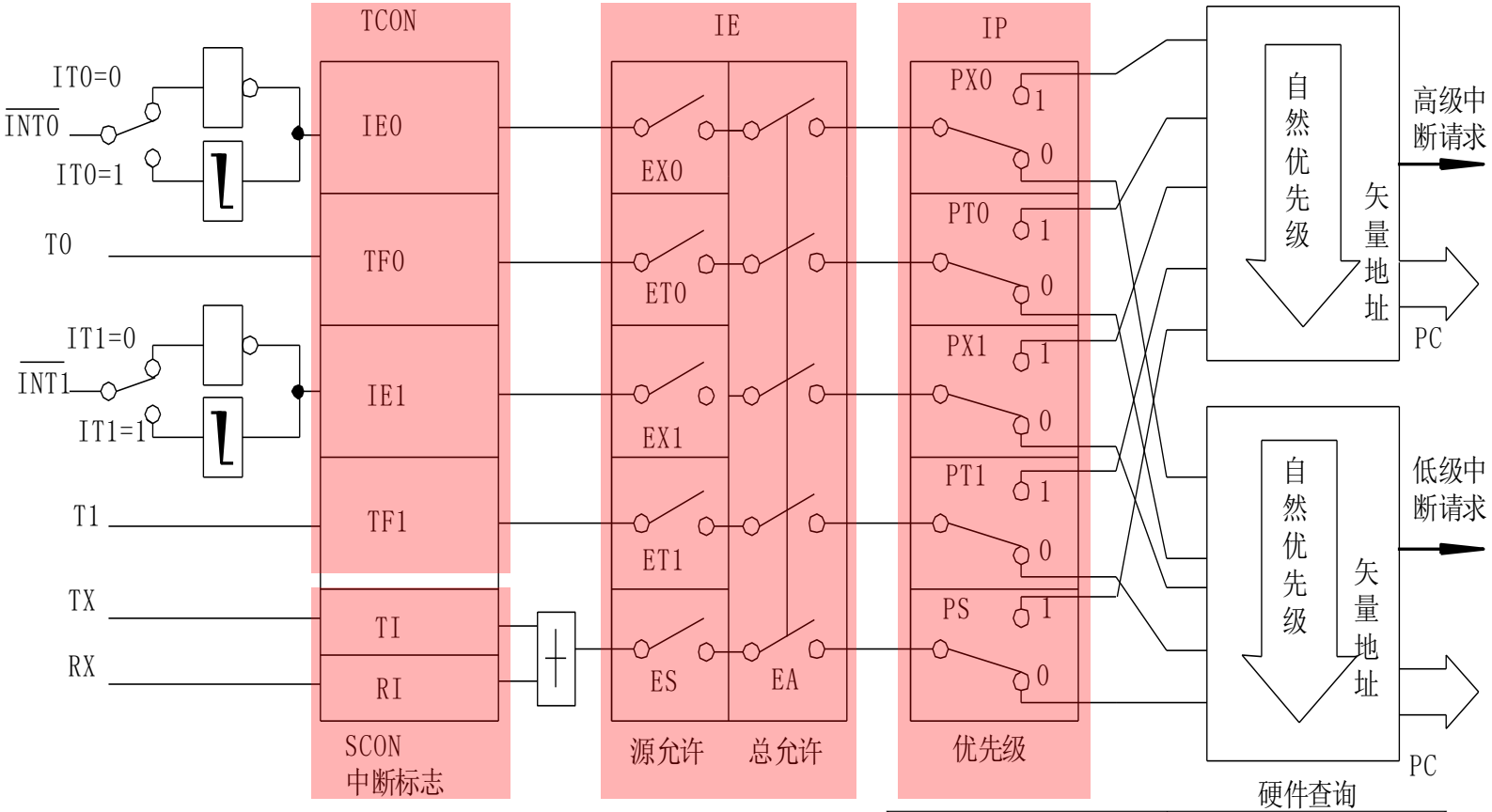
	0	0	0	0	0	0	0	0	Value after Reset
TCON	TF1	TR1	TF0	TR0	IE1	IT1	IE0	IT0	Bit name
	bit7	bit6	bit5	bit4	bit3	bit2	bit1	bit0	

- Count for what?





8051 interrupts (hardware)



F8h-FFh	(F8h-FFh)				
F0h-F7H	B (F0h-F7H)				
E8h-EFh	(E8h-EFh)				
E0h-E7h	ACC (E0h-E7h)				
D8h-DFh	(D8h-DFh)				
D0h-D7h	PSW (D0h-D7h)				
C8h-CFh	(C8h-CFh)				
C0h-C7h	(C0h-C7h)				
B8h-BFh	IP (B8h-BFh)				
B0h-B7h	P3 (B0h-B7h)				
A8h-AFh	IE (A8h-AFh)				
A0h-A7h	P2 (A0h-A7h)				
98h-9Fh	SCON (98h-9Fh)	SBUF			
90h-97h	P1 (90h-97h)				
88h-8Fh	TCON (88h-8Fh)	TMOD	TL0	TL1	TH0
80h-87h	P0 (80h-87h)	SP	DPL	DPH	
30h-7Fh					

TCON(88H)： 定时控制寄存器
SCON(98H)： 串行口控制寄存器
IE(A8H) ： 中断允许寄存器
IP(B8H) ： 中断优先级寄存器

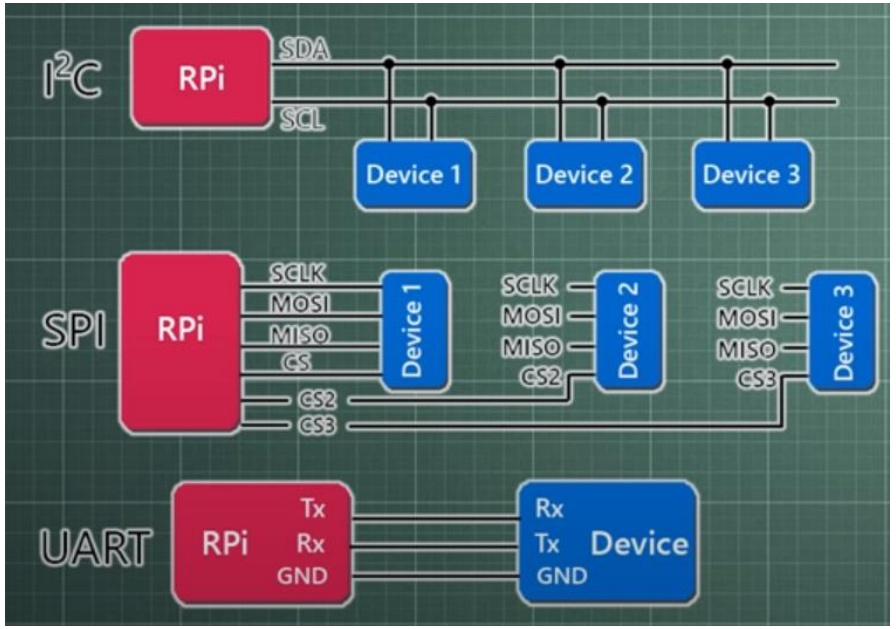
硬件查询

中断源	中断向量
上电复位	0000H
外部中断 0	0003H
定时器 0 溢出	000BH
外部中断 1	0013H
定时器 1 溢出	001BH
串行口中断	0023H
定时器 2 溢出	002BH (8052+)

- ORG 0000H
- LJMP MAIN ;跳过中断地址表，转主程序
- ORG 0003H
- LJMP INT0F ;转INT0中断服务
- ORG 000BH
- LJMP T0F ;转T0中断服务
- ORG 0013H
- LJMP INT1F ;转INT1中断服务
- ORG 001BH
- LJMP T1F ;转T1中断服务
- ORG 0023H
- LJMP SIOF ;转串口中断服务
- ORG 0030H
- MAIN: MOV SP, #5FH ;主程序

Communication Summary

- Wired communication
  - Parallel: CPU and RAM
    Serial: most peripherals
  - Synchronous vs. asynchronous
  - Simplex or duplex or half-duplex
  - In terms of distance
    RS232/RS485: long distance
    I2C, SPI: inter-chip
  - Principle and how to use
    - UART
    - I2C (Open-drain output & wired logic)
    - SPI



UART	I2C	SPI
1 to 1	Half-Duplex	Duplex
Asynchronous	Synchronous	Synchronous
2 wires	2 wires	4+ wires
20Kbps	1Mbps	25Mbps
Acknowledge pins	NO acknowledge	NO acknowledge
Medium power	Medium power	Lower power
15m distance	1m distance	20cm distance